Relatório 9 - Prática: Git e Github para Iniciantes (III)

Matheus Beuren Carvalho

1. Introdução

A tarefa designada neste card é a introdução ao Git e Github, ferramentas que são utilizadas no LAMIA, como especificado no card. Durante este relatório, serão abordados conhecimentos básicos e avançados de versionamento com git, o Github e também ao final, conteúdo referente ao Github Desktop.

2. Desenvolvimento

O principal problema ao não usar Git, é a necessidade de fazer várias cópias de um projeto ou arquivo que precisa de modificações ou novas versões. Esta ferramenta, vem para solucionar este problema, que ocupa muito espaço do hd, e assim fazendo cópias desnecessárias na sua máquina.

Diferente de outras ferramentas que, ao colocar uma nova versão só retornava o que mudava de uma para outra, tal atitude não é vantajosa para um projeto. O Git, tira como se fosse um print dos estados atuais(snapshots), levando assim todos os arquivos modificados e não modificados.

O Github é um serviço web compartilhado para projetos que utilizam o git como versionamento. Ele disponibiliza repositórios, sendo em teoria, como uma pasta onde você pode verisonar o seu projeto com Git. É uma das ferramentas mais utilizadas, senão a mais utilizada no mundo de desenvolvimento, pois ela une códigos do seu projeto e também facilita muito o trabalho em equipe em projetos.

Primeiro vídeo na prática, o instrutor ensina a instalar o Git e os comandos de configuração, como cadastrar email, username e etc. Neste caso, já possuía em minha máquina o Git instalado e configurado. Já em outra seção do minicurso, fizemos a inicialização do repositório ao criar uma pasta para tal.

Seguindo, é ensinado a escrever pelo editor do terminal, eu utilizei o nano para abrir um editor, e depois mostrei o conteúdo com o cat.

O status do arquivo em um repositório versionado pelo Git, é essencial para entender o versionamento em si. Para isso, é separado em quatro status de arquivo diferentes, sendo eles:

- Untracked: arquivo novo que ainda não foi versionado pelo Git;
- Unmodified: é quando o arquivo existe e foi versionado, porém não foi modificado;
- Modified: significa que o arquivo versionado foi modificado;
- Staged: mostra os arquivos que selecionou para versionar. Após o commit, os arquivos voltam a serem Unmodified.

Esse fluxo de status se repete a cada novo arquivo criado ou modificado no repositório.

Um comando muito importante, que também é utilizado para outras ocasiões, mas aqui no Git é crucial. Os Logs são registros gerados pelo sistema, mostrando o que foi feito, alterado e quem fez, através dos comandos git log e git show

O git diff, é muito importante para ver a diferença entre a modificação e a última versão de um arquivo, ou um commit.

Uma parte essencial foi a de voltar commits e modificações em arquivos: o git checkout retorna o arquivo para a última versão, o git reset HEAD desfaz alterações antes do staged, e o git reset --mixed move um commit selecionado de volta para modified.

Em outra seção do curso, aprendemos a criar um repositório no github, criar e adicionar uma chave SSH, ligar um repositório local e um remoto, clonar repositórios e a fazer fork em repositórios.

À primeira vista, é apresentado como criar um repositório. Em seguida, criamos uma chave SSH e adicionamos pelo terminal esta chave no github para que assim não ter problemas com repositório remoto.

O instrutor ensina a conectar um repositório local e um remoto criando um no github, porém já tenho criado um repositório para o Bootcamp e todos os comandos serão feitos na pasta “card-9”, assim não criarei outro repositório mas sim usar o criado. Nisso, o instrutor ensina o ‘push’ que seria a parte referente a mandar os commits feitos no local para o repositório remoto no github.

O git clone, é importante para copiar o repositório próprio ou de outra pessoa, ele permite pegar todo o conteúdo do repositório, mas somente pode ser feito o envio de alterações se o repositório for seu, senão somente será copiado.

O fork tem quase o mesmo papel que o clone, porém você pode fazer alterações e mandar para o usuário que é dono do repositório.

As branches, são um dos principais recursos no github, sendo elas outros ponteiros que apontam para outros commits. A principal vantagem de utilizá-la, é a facilidade de trabalho em grupo, com muitas pessoas mexendo no código, evitando conflitos.Na prática, criei uma branch com git checkout -b, trocando branches com git checkout e removi a branch com git branch -D depois de usar.

Existem dois conceitos muito importantes quando trabalhamos com git e github, que são o merge e o rebase. São dois conceitos parecidos mas com soluções diferentes. Começando pelo merge, ele junta a branch criada para a main, isso transforma tudo em ordem, porém precisa de um commit para fazer isso e por isso tem como desvantagem o histórico de commits, deixando-o poluído. E o rebase invés de colocar o commit em ordem, ele coloca no início da fila, ou seja, os commits da branch que vai para main, vão todos para o início, nisso sua principal desvantagem é a ordem dos commits, que trabalhando em equipe pode gerar muitos conflitos.

Figura 1 - Git Merge

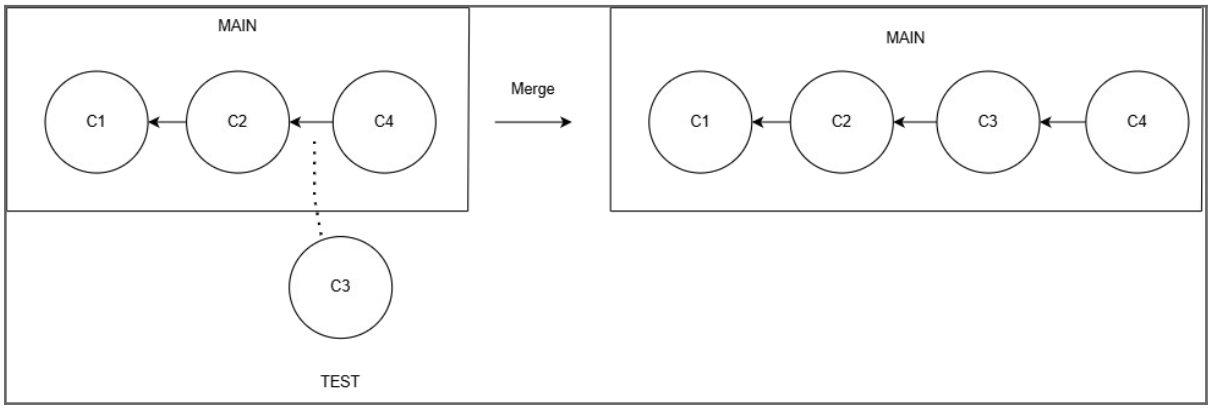
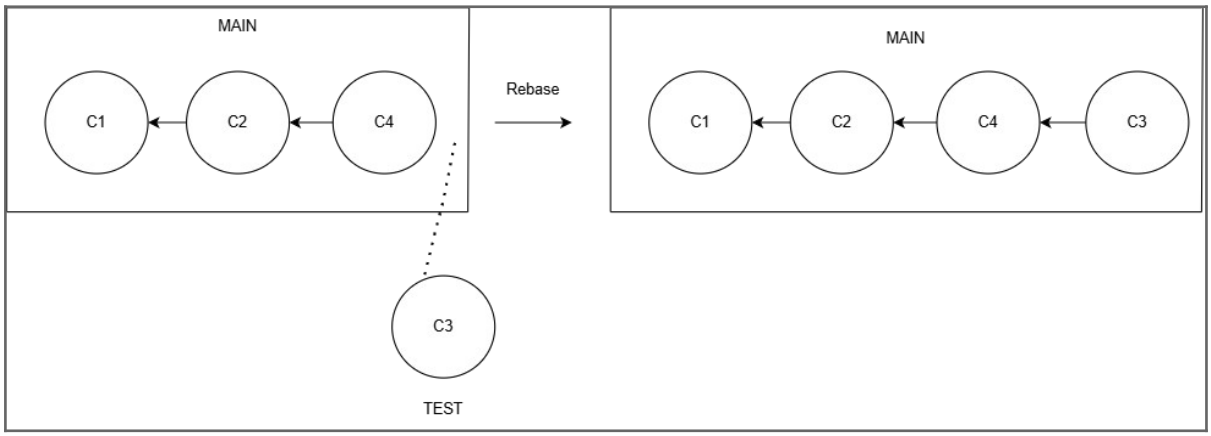


Figura 2 - Git Rebase

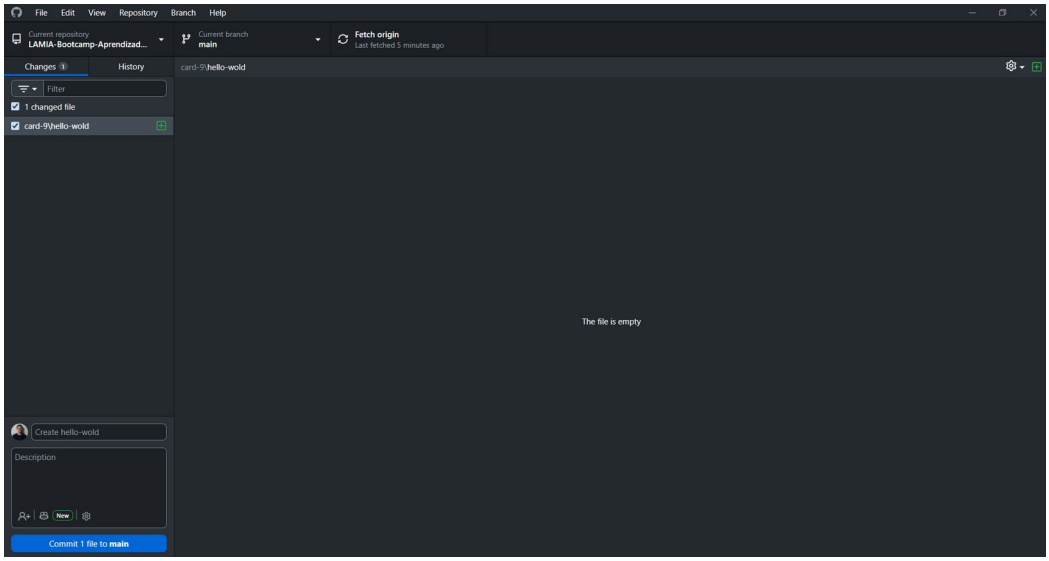


O merge foi feito assim, unindo a branch de teste à main em um novo commit. Já o rebase reorganizou o histórico, colocando os commits da branch testada no início da sequência, como mostra o git log --graph.

Finalizando este minicurso, foi visto não apenas duas ferramentas qualquer, mas sim ferramentas que serão utilizadas a todo momento em projetos. Desde os conceitos mais básicos, de status de arquivo para merge e rebase, todos eles são essenciais para a carreira de todo desenvolvedor.

Por fim, foi designado pelo card assistir a aula de github desktop, que por sua vez, facilita todo esse processo. O Github Desktop, torna todo esse processo de comandos via terminal muito mais fácil por ter uma interface amigável e dinâmica, podendo da mesma forma, fazer commit, adicionar, ver os arquivos modificados e o que mudou neles, dar push para o repositório e muito mais, conforme visto na (**Figura 3).**

Figura 3 - Github Desktop



3. Conclusão

A combinação Git e Github, resolve o problema de versionamento em arquivos e pastas, não sendo necessário criar toda vez uma pasta diferente para cada versão nova. Além disso, resolve o problema de trabalho em equipe, onde pode-se trabalhar com uma ou mais pessoas no mesmo repositório. A manipulação de comandos Git e o conhecimento do github, é estritamente necessário para projetos no laboratório como citado no card, nisso esse minicurso deu a base para o uso desta ferramenta.